Section 12 : Pointers and References

Information

- Title: `Section 12 : Pointers and References`
- Authors: `Etienne KOA`

Aperçu

- Qu'est-ce qu'un pointeur ?
- Déclarer des pointeurs
- Stockage des adresses dans des pointeurs
- Pointeurs de déréférencement
- Allocation dynamique de mémoire
- Arithmétique du pointeur
- Pointeurs et tableaux
- Passage par référence avec des pointeurs
- const et pointeurs
- Utiliser des pointeurs vers des fonctions
- Pièges potentiels du pointeur
- Qu'est-ce qu'une référence ?
- Réviser les références de passage aux fonctions
- Const et références
- Variables de référence dans les boucles for basées sur une plage
- Pièges potentiels des références
- Pointeurs bruts ou intelligents

Qu'est-ce qu'un pointeur ?

- Une variable
  - dont la valeur est une adresse
- Que peut-il y avoir à cette adresse ?
  - Une autre variable
  - Une fonction
- Les pointeurs pointent vers des variables ou des fonctions ?
- Si `x` est une variable entière et que sa valeur est 10 alors je peux déclarer un pointeur vers elle.
- Pour utiliser les données vers lesquelles pointe le pointeur, vous devez connaître son type.

Pourquoi utiliser des pointeurs ?

Ne puis-je pas simplement utiliser la variable ou la fonction elle-même ?

Oui, mais pas toujours

- À l'intérieur des fonctions, des pointeurs peuvent être utilisés pour accéder aux données définies en dehors de la fonction. Ces variables peuvent ne pas être dans la portée, vous ne pouvez donc pas y accéder par leur nom.
- Les pointeurs peuvent être utilisés pour fonctionner sur des tableaux de manière très efficace
- Nous pouvons allouer de la mémoire dynamiquement sur le tas ou sur le magasin gratuit
  - Cette mémoire n'a même pas de nom de variable.
  - La seule façon d'y accéder est via un pointeur
- Avec `OO`. les pointeurs sont le fonctionnement du polymorphisme !
- Peut accéder à des adresses spécifiques en mémoire
  - utile dans les applications embarquées et systèmes

Déclarer des pointeurs

```
variable_type *pointer_name;
```

```
int *int_ptr;
double* double_ptr;
char *char_ptr;
string *string_ptr;
```

Figure 1: Pointeur

Initialisation des variables de pointeur pour «pointer nulle part»

- Toujours initialiser les pointeurs
- Les pointeurs non initialisés contiennent des données inutiles et peuvent « pointer n'importe où »
- L'initialisation à zéro ou `nullptr(C++ 11)` représente l'adresse zéro
  - Implique que le pointeur « ne pointe nulle part »

```
variable_type *pointer_name {nullptr};
```

```
int *int_ptr {};
double* double_ptr {nullptr};
char *char_ptr {nullptr};
string *string_ptr {nullptr};
```

Figure 2: Pointeur

- Si vous n'initialisez pas un pointeur pour qu'il pointe vers une variable ou une fonction, vous devez l'initialiser à «`nullptr` pour le « rendre nul ».

Accès à l'adresse du pointeur ?

- `&` l'opérateur d'adresse
- Les variables sont stockées dans des adresses uniques
- Opérateur unaire
- Évalue à l'adresse de son opérande
  - L'opérande ne peut pas être une constante ou une expression évaluée à des valeurs temporaires.

```
int num{10};

cout << "Value of num is: " << num << endl;      // 10

cout << "sizeof of num is: " << sizeof num << endl;  // 4

cout << "Address of num is: " << &num << endl;      // 0x61ff1c
```

Figure 3: Pointeur

- `&` l'opérateur d'adresse - Exemple
- `sizof` une variable de pointeur
- Ne confondez pas la taille d'un pointeur et la taille de ce qu'il ˆpointe
- Tous les pointeurs d'un programme ont la même taille
- Ils peuvent pointer vers des types très grands ou très petits

```
int *p;

cout << "Value of p is: " << p << endl; // 0x61ff60 - garbage

cout << "Address of p is: " << &p << endl; // 0x61ff18

cout << "sizeof of p is: " << sizeof p << endl; // 4

p = nullptr;  // set p to point nowhere

cout << "Value of p is: " << p << endl;     // 0
```

Figure 4: Pointeur

```
int *p1 {nullptr};
double *p2 {nullptr};
unsigned long long *p3 {nullptr};
vector<string> *p4 {nullptr};
string *p5 {nullptr};
```

Figure 5: Pointeur

Typed Pointers

- Le compilateur s'assurera que l'adresse stockée dans une variable de pointeur est du type correct.

```
int score {10};
double high_temp {100.7};

int *score_ptr {nullptr};

score_ptr = &score;     // OK

score_ptr = &high_temp; // Compiler Error
```

Figure 6: Pointeur

- `&` l'opérateur d'adresse
- Les `Pointeurs` sont des variables donc ils peuvent changer
- Les `pointeurs` peuvent être nuls
- Les « pointeurs » peuvent être non initialisés

```
double high_temp {100.7};
double low_temp {37.2};

double *temp_ptr;

temp_ptr = &high_temp; // points to high_temp
temp_ptr = &low_temp;  // now points to low_temp

temp_ptr = nullptr;
```

Figure 7: Pointeur

Déférence d'un pointeur

Accédez aux données vers lesquelles nous pointons - en déférant un pointeur

- Si `score_ptr` est un pointeur et a une adresse valide
- Ensuite, vous pouvez accéder aux données à l'adresse contenue dans le `score_ptr` en utilisant l'opérateur de déférence `*`

```
int score {100};
int *score_ptr {&score};

cout << *score_ptr << endl;       // 100

*score_ptr = 200;
cout << *score_ptr << endl;       // 200
cout << score << endl;         // 200
```

Figure 8: Pointeur

- Accédez aux données vers lesquelles nous pointons

```
double high_temp {100.7};
double low_temp {37.4};
double *temp_ptr {&high_temp};

cout << *temp_ptr << endl;    // 100.7

temp_ptr = &low_temp;

cout << *temp_ptr << endl;    // 37.4
```

```
string name {"Frank"};

string *string_ptr {&name};

cout << *string_ptr << endl;      // Frank

name = "James";

cout << *string_ptr << endl;      // James
```

Allocation de mémoire dynamique

Stockage d'allocation à partir du tas au moment de l'exécution

- Souvent, nous ne savons pas de quelle quantité de stockage nous avons besoin jusqu'à ce que nous en ayons besoin
- Nous pouvons allouer du stockage pour une durée et une durée d'exécution variables
- Rappeler les tableaux `C++`
  - Nous avons dû fournir explicitement la taille et cela a été corrigé
  - Mais les vecteurs grandissent et rétrécissent dynamiquement
- Nous pouvons utiliser des pointeurs pour accéder au stockage de tas nouvellement alloué

utiliser `new` pour allouer du stockage

```
int *int_ptr {nullptr};

int_ptr = new int;          // allocate an integer on the heap

cout << int_ptr << endl;   // 0x2747f28

cout << *int_ptr << endl;  // 41188048 - garbage

*int_ptr = 100;

cout << *int_ptr << endl;  // 100
```

Figure 9: Pointeur

en utilisant `delete` pour désallouer le stockage

utiliser `new[]` pour allouer du stockage d' un tableau

utiliser `delete[]` pour libérer le stockage d'un tableau

```
int *int_ptr {nullptr};

int_ptr = new int;    // allocate an integer on the heap

. . .

delete int_ptr;    // frees the allocated storage
```

Figure 10: Pointeur

```
int *array_ptr {nullptr};
int size {};

cout << "How big do you want the array? ";
cin >> size;

array_ptr = new int[size];  // allocate array on the heap

// We can access the array here
// we'll see how in a few slides
```

Figure 11: Pointeur

```
int *array_ptr {nullptr};
int size {};

cout << "How big do you want the array?";
cin >> size;

array_ptr = new int[size];  // allocate array on the heap

. . .

delete [] array_ptr;    // free allocated storage
```

Figure 12: Pointeur

Relation entre les tableaux et les pointeurs

- La valeur d'un nom de tableau est l'adresse du premier élément du tableau
- La valeur de la variable pointeur est une adresse
- Si le pointeur pointe vers le même type de données que l'élément du tableau, alors le pointeur et le nom du tableau peuvent être utilisés de manière interchangeable (presque).

```
int scores[] {100, 95, 89};

cout << scores << endl;     // 0x61fec8
cout << *scores << endl;    // 100

int *score_ptr {scores};

cout << score_ptr << endl;  // 0x61fec8

cout << *score_ptr << endl; // 100
```

```
int scores[] {100, 95, 89};

int *score_ptr {scores};

cout << score_ptr[0] << endl;  // 100

cout << score_ptr[1] << endl;  // 95

cout << score_ptr[2] << endl;  // 89
```

Utilisation de pointeurs dans les expressions

```
int scores[] {100, 95, 89};

int *score_ptr {scores};

cout << score_ptr << endl;      // 0x61ff10

cout << (score_ptr + 1) << endl; // 0x61ff14

cout << (score_ptr + 2) << endl; // 0x61ff18
```

```
int scores[] {100, 95, 89};

int *score_ptr {scores};

cout << *score_ptr << endl;    // 100

cout << *(score_ptr + 1) << endl;  // 95

cout << *(score_ptr + 2) << endl;  // 89
```

Équivalence des notations indice et décalage

```
int array_name[] {1,2,3,4,5};

int *pointer_name {array_name};
```

Subscript Notation	Offset Notation
array_name[index]	*(array_name + index)
pointer_name[index]	*(pointer_name + index)

Figure 13: Pointeur

Arithmétique du pointeur

- Les pointeurs peuvent être utilisés dans
  - Expressions d'affectation
  - Expressions arithmétiques
  - Expressions de comparaison
- `C++` permet l'arithmétique des pointeurs
- L'arithmétique des pointeurs n'a de sens qu'avec les tableaux bruts

`++` et `--`

- `(++)` : incrémente un pointeur pour pointer vers l'élément suivant du tableau `int_ptr++` ;
- `(--)` : décrémente un pointeur pour pointer vers l'élément précédent du tableau `int_ptr--` ;

`+` et `-`

- `(+)` : incrémente le pointeur de `n * sizeof(type)`

`int_ptr += n;` ou `int_ptr = int_ptr + n;`

- `(-)` : décrémenter le pointeur de `n * sizeof(type)`

`int_ptr -= n;` ou `int_ptr = int_ptr - n;`

Soustraire deux pointeurs

- Déterminer le nombre d'éléments entre les pointeurs
- Les deux pointeurs doivent pointer vers le même type de données

```
int n = int_ptr2 - int_ptr1 ;
```

Comparaison de deux pointeurs `==` et `!=`

- Déterminer si deux pointeurs pointent vers le même emplacement
  - ne compare PAS les données là où elles pointent !

```
string s1 {"Frank"};
string s2 {"Frank"};

string *p1 {&s1};
string *p2 {&s2};
string *p3 {&s1};

cout << (p1 == p2) << endl;   // false
cout << (p1 == p3) << endl;   // true
```

Figure 14: Pointeur

La comparaison des pointeurs de données pointe vers

- Déterminer si deux pointeurs pointent vers les mêmes données
  - vous devez comparer les pointeurs référencés

```
string s1 {"Frank"};
string s2 {"Frank"};

string *p1 {&s1};
string *p2 {&s2};
string *p3 {&s1};

cout << (*p1 == *p2) << endl;   // true
cout << (*p1 == *p3) << endl;   // true
```

Figure 15: Pointeur

Const et Pointeurs

const et pointeurs

- Il existe plusieurs façons d'auqlifier les pointeurs en utilisant `const`
  - Pointeurs vers des constantes
  - Pointeurs constants
  - Pointeur constant vers des constantes

Pointeurs vers des constantes

- Les données pointées par les pointeurs sont constantes et ne peuvent pas être modifiées
- Le pointeur lui-même peut changer et pointer ailleurs.

```
int high_score {100};
int  low_score { 65};
const int *score_ptr { &high_score };

*score_ptr = 86;      // ERROR
score_ptr = &low_score; // OK
```

Figure 16: Pointeur

Pointeurs constants

- Les données pointées par les pointeurs peuvent être modifiées.
- Le pointeur lui-même « ne peut pas » changer et pointer ailleurs.

```
int high_score {100};
int  low_score { 65};
int  *const score_ptr { &high_score };

*score_ptr = 86;      // OK
score_ptr = &low_score; // ERROR
```

Figure 17: Pointeur

Pointeurs constants vers des constantes

- Les données pointées par le pointeur sont constantes et « ne peuvent pas » être modifiées.
- Le pointeur lui-même « ne peut pas » changer et pointer ailleurs.

Renvoyer un pointeur depuis une fonction

- Les fonctions peuvent également renvoyer des pointeurs

```
type *function();
```

```
int high_score {100};
int  low_score { 65};
const int *const score_ptr { &high_score };

*score_ptr = 86;      // ERROR
score_ptr = &low_score; // ERROR
```

Figure 18: Pointeur

- Devrait renvoyer des pointeurs vers
  - Mémoire allouée dynamiquement dans la fonction
  - Aux données transmises
- Ne renvoie jamais de pointeur vers une variable de fonction locale

Passage par référence avec pointeurs - définition de la fonction

```
void double_data(int *int_ptr);

void double_data(int *int_ptr) {
   *int_ptr *= 2;

   // *int_ptr = *int_ptr * 2;
}
```

Figure 19: Pointeur

Passage par référence avec pointeurs - appel de la fonction

```
int main() {
   int value {10};

   cout << value << endl;     // 10

   double_data( &value);

   cout << value << endl;     // 20
}
```

Figure 20: Pointeur

Renvoyer un paramètre

```
int *largest_int(int *int_ptr1, int *int_ptr2) {
   if (*int_ptr1 > *int_ptr2)
      return int_ptr1;
   else
      return int_ptr2;
}

int main() {
   int a{100};
   int b{200};

   int *largest_ptr {nullptr};
   largest_ptr = largest_int(&a, &b);
   cout << *largest_ptr << endl;   // 200
   return 0;
}
```

Renvoi de la mémoire allouée dynamiquement

```
int *create_array(size_t size, int init_value = 0) {

   int *new_storage {nullptr};

   new_storage = new int[size];
   for (size_t i{0}; i < size; ++i)
      *(new_storage + i) = init_value;
   return new_storage;
}

int main() {
   int *my_array;    // will be allocated by the function

   my_array = create_array(100,20);  // create the array

    // use it

   delete [] my_array; /    / be sure to free the storage
   return 0;
}
```

Ne renvoyez jamais un pointeur vers une variable locale !!

Pièges potentiels des pointeurs

- Pointeurs non initialisés
- Pointeurs pendants
- Ne vérifie pas si le nouveau n'a pas réussi à allouer de la mémoire

```
int *dont_do_this () {
   int size {};
   . . .
   return &size;
}
int *or_this () {
   int size {};
   int *int_ptr {&size};
   . . .
   return int_ptr;
}
```

Figure 21: Pointeur

- Fuite de mémoire

Pointeurs non initialisés

```
int *int_ptr; // pointing anywhere

. . .

*int_ptr = 100; // Hopefully a crash
```

Figure 22: Pointeur

```
int * int_ptr; // pointant n'importe où

...

*int_ptr = 100 ; // Espérons un crash
```

Pointeur pendant

- Pointeur qui pointe vers la mémoire libérée
  - Par exemple, 2 pointeurs pointent vers les mêmes données
  - 1 pointeur libère les données avec suppression
  - L'autre pointeur accède aux données de version
- Pointeur qui pointe vers une mémoire invalide
  - Nous avons vu cela lors du retour d'un pointeur vers une variable locale de fonction

Ne pas vérifier si `new` a échoué

- Si `new` échoue, une exception est levée
- Nous pouvons utiliser la gestion des exceptions pour détecter les exceptions

- Le déréférencement d'un pointeur nul entraînera le crash de votre programme.

Fuite de mémoire

- Oublier de libérer la mémoire allouée avec suppression
- Si vous perdez le pointeur vers le stockage alloué sur le tas, vous n'avez aucun moyen d'accéder à nouveau à ce stockage.
- La mémoire est orpheline ou a fui
- L'un des problèmes de pointeur les plus courants

Qu'est-ce qu'une référence ?

- Un alias pour une variable
- Doit être initialisé à une variable lors de sa déclaration
- Ne peut pas être nul
- Une fois initialisé, il ne peut pas être fait référence à une variable différente
- Très utile comme paramètres de fonction
- Il pourrait être utile de considérer une référence comme un pointeur constant automatiquement déréférencé

Utilisation de références dans une boucle `for` basée sur une plage

```
vector<string> stooges {"Larry", "Moe", "Curly"};

for (auto str: stooges)
      str = "Funny";       // changes the copy

for (auto str:stooges)
    cout << str << endl;      // Larry, Moe, Curly

vector<string> stooges {"Larry", "Moe", "Curly"};

for (auto &str: stooges)
      str = "Funny";       // changes the actual

for (auto str:stooges)
    cout << str << endl;      // Funny, Funny, Funny

vector<string> stooges {"Larry", "Moe", "Curly"};

for (auto const &str: stooges)
      str = "Funny";    // compiler error
```

```
vector<string> stooges {"Larry", "Moe", "Curly"};

for (auto const &str:stooges)
    cout << str << endl;      // Larry, Moe, Curly
```

- Passer des références aux fonctions
  - Merci de vous référer à la section 11

Valeurs L et Valeurs R

Valeurs L + valeurs qui ont des noms et sont adressables + modifiables si ce ne sont pas des constantes

```
int x {100};   // x is an l-value
x = 1000;
x = 1000 + 20;

string name;   // name is an l-value
name = "Frank";

100 = x;           // 100 is NOT an l-value
(1000 + 20) = x;  // (1000 + 20) is NOT an l-value

string name;
name = "Frank";
"Frank" = name;  // "Frank" is NOT an l-value
```

Valeurs R

- Valeur R (non adressable et non cessible)
  - Une valeur qui n'est pas une valeur L
    - à droite d'une expression d'affectation
    - un littéral
    - un temporaire qui se veut non modifiable
- Les valeurs R peuvent être attribuées explicitement aux valeurs L

références de valeur l + Les références que nous avons utilisées sont des références de valeur L + Parce que nous faisons référence aux valeurs L

- Pareil quand on passe par référence

Récapitulatif de la section

1) Quand utiliser les paramètres de pointeurs ou de références

```
int x {100};                    // 100 is an r-value
int y = x + 200;               // (x+200)is an r-value

string name;
name = "Frank";              // "Frank" is an r-value

int max_num = max(20,30); // max(20,30) is an r-value
```

Figure 23: Pointeur

```
int x {100};
int y {0};

y = 100;     // r-value 100 assigned to l-value y

x = x + y;   // r-value (x+y) assigned to l-value x
```

Figure 24: Pointeur

```
int x {100};

int &ref1 = x;    // ref1 is reference to l-value
ref1 = 1000;

int &ref2 = 100; // Error 100 is an r-value
```

Figure 25: Pointeur

```
int square(int &n) {
   return n*n;
}

int num {10};

square(num);   // OK

square(5);   // Error - can't reference r-value 5
```

Figure 26: Pointeur

- Passage par valeur
  - Lorsque la fonction ne modifie “pas” le paramètre réel et
  - le paramètre est petit et efficace pour copier comme de simples types (int, char, double, etc.)
- Passage par référence à l’aide d’un pointeur
  - Lorsque la fonction “modifie” le paramètre réel, et
  - le paramètre coûte cher à copier, et
  - c’est OK pour le pointeur, une valeur `nullptr` est autorisée
- Passage par référence utilisant un pointeur `const` vers `const`
  - Lorsque la fonction ne modifie “pas” le paramètre réel et
  - le paramètre coûte cher à copier, et
  - c’est OK sur le pointeur, on autorise une valeur `nullptr`, et
  - Vous ne souhaitez pas modifier le pointeur lui-même
- Passage par référence à l’aide d’une référence
  - Lorsque la fonction modifie le paramètre réel, et
  - le paramètre coûte cher à copier, et
  - c’est OK pour le pointeur, une valeur `nullptr` est autorisée
- Passage par référence utilisant une référence `const`
  - Lorsque la fonction ne modifie “pas” le paramètre réel et
  - le paramètre coûte cher à copier, et
  - c’est OK pour le pointeur, on autorise un `nullptr`